

Introduction to Retrieval-Augmented Generation

These notes are the sample document for the RAG intro project.

The indexer splits this PDF into overlapping chunks, embeds each chunk with OpenAI text-embedding-3-large, and stores the vectors in Qdrant.

The FastAPI query route retrieves the nearest chunks and asks an LLM to answer only from that context, citing page numbers.

What is RAG?

Retrieval-Augmented Generation combines a search step with a language model. Instead of asking the model to recall facts from training data, the application first finds relevant passages from a private knowledge base and then passes those passages to the model as context.

The stack used in this project

Qdrant is the vector database. Docker Compose starts qdrant/qdrant and publishes port 6333. This project uses collection sample_collection.

OpenAI embeddings and chat

Indexing uses text-embedding-3-large. Query answering uses a chat model set by OPENAI_CHAT_MODEL. Both read OPENAI_API_KEY from the environment.

FastAPI query route

POST /query/ accepts a question. The handler runs similarity_search against Qdrant and sends a system prompt plus the user question.

How to run the demo

Start Qdrant with docker compose up -d.

Copy .env.example to .env and put in a valid OpenAI API key.

POST /index/ a PDF or run python index.py.

Start the API with python -m uvicorn main:app --reload.

Facts you can ask about

Qdrant listens on port 6333.

The embedding model is text-embedding-3-large.

Chunks are 1000 characters with 400 characters of overlap.

The collection name is sample_collection.

The query router prefix is /query.